OpCode-Stats: A Statistical Analysis of Compiled Program Instruction Sequences

Ara Aslyan
Strike Technologies, New York
aaslyan@striketechnologies.com

Abstract

Compiled binary executables, when viewed as flat sequences of machine opcodes, exhibit rich statistical structure that shares properties observed in natural language and biological sequences. We present *OpCode-Stats*[1], an open-source toolkit (inspired by computational biology techniques applied to DNA sequences, hence the name *Binary DNA*) that applies computational biology and information-theoretic techniques to the systematic study of program instruction sequences. Across two corpora totalling up to 4.65 million instructions, we characterise opcode frequency distributions, n-gram entropy rates, compressibility, recurring motifs, and manifold dimensionality, and validate that the measured statistics are stable as corpus size grows. The toolkit and all experimental data are publicly available.

1 Introduction

The last decade has witnessed remarkable progress in language models trained on source code [1, 3]. Yet the statistical properties that make code so learnable remain poorly characterised at the machine-instruction level. Source code is parsed into abstract syntax trees, compiled through multiple transformation passes, and ultimately reduced to a sequence of opcodes—a representation that discards all high-level structure yet, as we show here, retains a profound statistical regularity.

Bioinformatics offers a natural set of tools for studying such sequences. DNA and protein sequences are studied with k-mer analysis, entropy profiling, motif discovery, and information-theoretic metrics [18]. We propose that binary instruction sequences—hereafter *opcode sequences*—are amenable to the same treatment.

Contributions. This paper makes the following contributions:

1. A reproducible, open-source analysis toolkit (*OpCode-Stats*, https://github.com/OpCode-Stats/OpCode-Stats) that applies computational biology techniques to compiled binaries.

2. Empirical characterisation of opcode frequency concentration: a heavy- tailed rank-frequency distribution (MLE $\hat{\alpha} \approx 1.4$; KS $= 0.114$) with a unigram–sequential decomposition of compression gains showing that 69% of compressibility advantage over random sequences comes from frequency skew and 31% from sequential structure.

3. Quantification of n-gram entropy structure and its deviation from unigram-shuffled baselines, revealing multi-scale sequential dependencies.

4. Systematic motif discovery linking recurrent k-mers to specific compiler-generated idioms.

5. A thin-manifold characterisation of the program space via compression, mutual information decay, and corpus-level PCA.

6. A controlled compiler matrix experiment (5 projects $\times$ {gcc, clang} $\times$ {`-O0`, `-O2`, `-O3`}) showing that source-code identity explains more NCD similarity than compiler identity ($p_{\text{project}} < 10^{-4}$, $r = 0.51$) and that compiler variation adds new manifold dimensions (d_{95} doubles from 8 to 17 when compiler axis is included).

7. Empirical validation that n-gram language model perplexity closely tracks the empirical conditional entropy (bigram gap < 0.07 bits), closing the loop between statistical structure and model performance.

The rest of the paper is organised as follows. Section 2 reviews related work. Section 3 describes our pipeline. Section 4 presents empirical findings. Section 5 interprets results in the context of language model performance. Section 6 concludes.

[1] https://github.com/OpCode-Stats/OpCode-Stats

2 Background and Related Work

2.1 Opcode-Level Program Analysis

Binary analysis for security and malware detection has a long history of exploiting opcode frequency features [2, 13]. Bilar [2] observed Zipf-like distributions in Windows PE files, noting that a small set of opcodes accounts for most instruction traffic. Subsequent work used n-gram opcode features for malware classification [13, 15], but focused on discriminative accuracy rather than the underlying information structure.

The closest prior cross-domain study is Gan et al. [7], who applied n-tuple Zipf analysis to DNA, English text, source code, and binary files. Our toolkit extends this comparison with a broader suite of information-theoretic measures (mutual information decay, LZ78 complexity, motif discovery, clone detection) and a controlled compiler-variation experiment. In the malware domain, MAlign [14] encodes raw binary bytes as nucleotides and applies genome sequence alignment to malware clustering, demonstrating the practical value of the biological sequence analogy for binary analysis.

Recent binary code embedding systems implicitly validate the existence of statistical structure in assembly but do not provide direct characterisation. PalmTree [11] learns BERT-style instruction embeddings from assembly corpora; jTrans [19] extends this with jump-aware Transformers for binary similarity detection. Both implicitly rely on the statistical regularities that OpCode-Stats makes explicit and measurable. Marcelli et al. [12] provide the most comprehensive systematisation of binary function similarity, evaluating approaches across 243K binaries and 26.8M functions; their benchmark is the natural target for future large-scale validation of OpCode-Stats's similarity modules. HermesSim [20] argues that graph-based semantic representations outperform NLP-style n-gram approaches for binary *similarity search*; this is orthogonal to our focus on statistical characterisation rather than similarity ranking. Nova [9], a generative language model for assembly, similarly demonstrates learnability but not measurement.

2.2 Compression-Based Similarity

Normalized Compression Distance (NCD) [4] provides a parameter-free measure of similarity grounded in Kolmogorov complexity. Its approximation via `zlib` or `lzma` has been applied to clustering music, genomes, and source files [4]. We apply NCD systematically to binary instruction sequences and compare it against n-gram TF-IDF similarity.

2.3 Information Theory of Programs

Shannon entropy and mutual information have been applied to source code to measure predictability [8, 17]. The *naturalness* hypothesis [8] states that software is more predictable than natural language because it is written by humans following conventions; cross-entropy of token-level language models on code is lower than on English prose. Our work extends this line to the binary level, measuring structural properties that survive compilation.

2.4 Motif Discovery in Sequences

K-mer based motif discovery originated in genomics for finding transcription factor binding sites [5]. In the binary domain, compiler idiom recognition has been studied in the context of decompilation and binary lifting [16], but systematic statistical characterisation of recurrent opcode patterns is absent from the literature.

3 Methodology

3.1 Corpus Design and Extraction

We analyse two corpora of GNU/Linux ELF x86-64 binaries drawn from `/usr/bin`:

- *Smoke* (15 binaries): grep, sed, gzip, bzip2, xz, find, tar, bash, python3, curl, git, vim, nano, gcc, objdump. Verified to have distinct inodes, eliminating hard-link duplicates.
- *Coreutils-system* (39 binaries after inode deduplication): drawn from 11 functional categories (text processing, compression, filesystem, networking, shells, editors, compilers, interpreters, version control, archives, utilities) with a 10 MB size cap. Adds awk, diff, zip, unzip, cp, mv, rm, chmod, chown, df, wget, ssh, scp, nc, dash, clang, as, ld, perl, tar, ar, strace, nm, readelf, addr2line to the smoke set.

Instruction sequences are extracted via `objdump -d` with a 60-second timeout per binary. Only the mnemonic of each instruction is retained; operands and addresses are discarded. This reduces each binary to a flat opcode sequence $s = (o_1, o_2, \ldots, o_N)$ where o_i belongs to a shared vocabulary $\mathcal{V}$. Functions are extracted separately to support per-function motif and positional analyses.

3.2 Frequency and Zipf Analysis

Let $f(v)$ denote the corpus-wide frequency of opcode $v \in \mathcal{V}$ and $r(v)$ its frequency rank. We fit Zipf's power law

$$f(v) \propto r(v)^{-\alpha} \tag{1}$$

via OLS on log-log coordinates. A 95% bootstrap confidence interval on α is obtained from 200 resamplings with replacement. As a complementary estimator, we also apply maximum likelihood estimation (MLE) to the truncated discrete Zipf model: we maximise $\ell(\alpha) = -\alpha \sum_r f(r) \log r - N \log Z_V(\alpha)$ where $Z_V(\alpha) = \sum_{r=1}^{V} r^{-\alpha}$ is the truncated Riemann zeta normalisation. A 95% CI on α_{MLE} is obtained via parametric bootstrap ($n = 200$ resamples from the fitted distribution), and a Kolmogorov–Smirnov statistic is computed against the fitted rank-CDF. We treat the KS statistic as a goodness-of-fit indicator, noting that the asymptotic critical value for $n = |\mathcal{V}|$ provides an approximate threshold; a rigorous parametric bootstrap p-value (Clauset et al., 2009) is left as future work.

3.3 N-gram Entropy and Shuffled Baseline

The *n-gram entropy* is

$$H_n = -\sum_{w \in \mathcal{V}^n} p(w) \log_2 p(w) \tag{2}$$

and the *entropy rate* is $h_r(n) = H_n/n$. We compute H_n for $n = 1, \ldots, 5$. To isolate the contribution of higher-order dependencies, we compare real rates against a *shuffled baseline*: each binary's opcode sequence is randomly permuted (preserving the unigram distribution), and entropy rates are recomputed. A negative *entropy gain* $\Delta h_r(n) = h_r^{\text{real}}(n) - h_r^{\text{shuffled}}(n) < 0$ for $n > 1$ confirms genuine multi-opcode dependencies.

3.4 Compression and LZ Complexity

We measure two practical proxies for Kolmogorov complexity:

1. **Compression ratio** $\rho = |C(s)|/|s|$ using `zlib` and `lzma` on the UTF-8 encoded opcode stream.
2. **LZ78 complexity** [21]: the number of distinct phrases K produced by the LZ78 factorisation of the integer-encoded opcode sequence of length N, reported as the normalised statistic $\tilde{c} = K/\log_2 N$ (phrases per log-bit of sequence length). For high-entropy sources, the expected phrase count scales as $K = \Theta(N/\log N)$, hence $\tilde{c} = \Theta(N/\log^2 N)$: absolute values grow with N and should be interpreted primarily *relative* to suitable baselines. More structured (more compressible) sequences produce substantially smaller K and $\tilde{c}$.

Both metrics are compared to two baselines:

- *Uniform-random baseline*: sequences of the same length drawn uniformly from $\mathcal{V}$ (ten replicates, averaged), capturing a maximum-entropy reference.
- *Unigram-shuffled baseline*: each binary's opcode sequence is randomly permuted (five replicates), preserving marginal frequencies while destroying sequential order. The gap between real and shuffled results isolates compression and complexity gains attributable to *sequential* structure beyond Zipfian skew.

Binaries that are significantly more compressible than both baselines exhibit redundancy inconsistent with a high-entropy source.

3.5 Motif Discovery

A *k-mer motif* is a tuple of k consecutive opcodes $m = (o_i, o_{i+1}, \ldots, o_{i+k-1})$. For each $k \in \{4, 5, \ldots, 12\}$, we enumerate all k-mers across all functions and retain motifs satisfying two thresholds:

- *Frequency*: appears in at least $\max(5, 0.005N_f)$ distinct functions, where N_f is the total function count.
- *Coverage*: appears in at least 1% of all functions.

Motifs are sorted by score freq $\times$ coverage and the top 100 per length are retained. Each motif is annotated with a semantic label (e.g. *function prologue*, *loop/branch*, *call sequence*) based on the opcodes present.

3.6 Positional Analysis

To characterise function boundary structure, we collect the opcode at each position $0, 1, \ldots, W-1$ from the start and end of every function with at least $2W$ instructions ($W = 20$). At each position we compute the empirical distribution over $\mathcal{V}$ and its Shannon entropy. Low entropy at a position indicates that a specific opcode appears there with high probability across functions, reflecting compiler convention.

3.7 Similarity, Clustering, and Dimensionality Reduction

We compute two families of pairwise binary similarity matrices:

1. **NCD**: $\mathrm{NCD}(x, y) = (C(xy) - \min(C(x), C(y)))/\max(C(x), C(y))$ with `zlib` and `lzma` as compressors C.
2. **N-gram cosine similarity**: L1-normalised term-frequency vectors over n-gram vocabularies for $n \in \{2, 3, 4\}$, with cosine distance as the metric. We use L1-normalised raw TF (no IDF weighting, `min_df`$= 1$) rather than TF-IDF, which is appropriate for small corpora where IDF penalties and document-frequency thresholds would otherwise discard the majority of features.

To quantify whether the NCD distance matrices capture category structure, we apply a one-sided Mann–Whitney U test comparing within-category to between-category pairwise distances (binaries with no category label excluded). Agglomerative hierarchical clustering (Ward, complete, and average linkage) is applied to each distance matrix and evaluated by cophenetic correlation. Two-dimensional embeddings are produced via PCA and UMAP. Cluster quality is measured by *silhouette score* and *category purity*: the fraction of cluster members sharing the majority category, averaged over clusters.

3.8 Information Geometry and Manifold Estimation

Sliding-window entropy. Shannon entropy is computed over windows of size $W \in \{16, 32, 64\}$ sliding along each binary's sequence, revealing local entropy variation.

Mutual information decay. We estimate $I(o_t; o_{t+\ell})$ for lags $\ell = 1, \ldots, 50$ using empirical joint and marginal distributions over a 50 000-instruction prefix. A shuffled baseline (same sequence, unigrams preserved) isolates the contribution of higher-order dependencies. The *MI half-life* is the smallest ℓ^* for which $I(\ell^*) \leq I(1)/2$.

Corpus manifold dimensionality. Each binary is represented by a normalised bigram frequency vector restricted to the top-200 most common bigrams. PCA on the resulting $B \times 200$ matrix (where B is the number of binaries) yields the number of components explaining 95% of inter-binary variance, d_{95}. A small ratio $d_{95}/200$ indicates that programs populate a thin sub-manifold of the theoretical bigram space.

3.9 N-gram Language Model Evaluation

To close the loop between the information-theoretic analysis and model-level predictions, we train simple n-gram language models for $n \in \{1, 2, 3, 4, 5\}$ with Laplace (add-1) smoothing. Each binary's opcode sequence is split 80/20 by token count; models are trained jointly on all binaries' training portions and evaluated independently on each binary's held-out portion.

For each n we measure per-token *cross-entropy*

$$H_{\mathrm{LM}}(n) = -\frac{1}{T} \sum_{t=1}^{T} \log_2 P_{\mathrm{LM}}\big(o_t \mid o_{t-n+1:t-1}\big) \quad (3)$$

and *perplexity* $\mathrm{PPL}(n) = 2^{H_{\mathrm{LM}}(n)}$. The theoretical lower bound on $H_{\mathrm{LM}}(n)$ is the empirical conditional entropy $H_n - H_{n-1}$, which equals the cross-entropy of a model with perfect frequency counts. The *smoothing gap* $\delta(n) = H_{\mathrm{LM}}(n) - (H_n - H_{n-1})$ quantifies the overhead from finite-sample estimation and Laplace smoothing.

3.10 Code Clone Detection

Compiler-generated code is replete with recurring instruction patterns: function prologues, epilogues, libc initialisation stubs, and common algorithm skeletons appear nearly verbatim across binaries. We detect *clone families*—groups of functions sharing substantial instruction-level similarity—using a three-stage pipeline.

Stage 1: MinHash signatures. Each function's opcode sequence is shingled with $k \in \{3, 4, 5\}$ to form a set of k-gram tokens. A MinHash signature of $m = 128$ permutations is computed for each function at each granularity, giving a Jaccard estimator with standard error $\leq 1/\sqrt{m} \approx 0.09$.

Stage 2: LSH candidate generation. Locality-Sensitive Hashing (LSH) is applied at Jaccard thresholds $\tau \in \{0.70, 0.90\}$ for each k. Candidate pairs from all six (threshold, k) configurations are merged; per-pair, the highest Jaccard estimate is retained. This reduces the comparison budget from $O(N^2)$ to $O(N \cdot C)$ where $C \ll N$ for typical corpora.

Stage 3: Smith-Waterman alignment. Each candidate pair undergoes Smith-Waterman local sequence alignment on opcode sequences. The normalised alignment score $s = S/\min(|f_1|, |f_2|)$ provides a more accurate similarity estimate than raw Jaccard. Pairs are classified by clone type:

- **Type 1** (exact/near-exact): $J \geq 0.95$—opcode sequences are identical up to trivial register renaming captured by the shingle vocabulary.
- **Type 2** (renamed/parameterised): $0.70 \leq J < 0.95$—same structure with different immediate constants or calling conventions.
- **Type 3** (gapped/structurally similar): $0.50 \leq J < 0.70$—shared algorithmic skeleton with insertions or deletions.

Clone pairs are assembled into *families* via connected components on the clone similarity graph. For each family we compute a *conserved core* (the intersection of all member opcode sets), a *divergence score* ($1 - \bar{J}$ across all intra-family pairs), and a *cross-binary* flag indicating whether members span more than one input binary.

4 Results

4.1 Corpus Statistics

Table 1 summarises both corpora after inode deduplication. The smoke corpus is a 15-binary pilot; the coreutils-system corpus scales to 39 binaries spanning 11 functional categories. Section 4.13 reports a direct comparison of all key metrics across the two corpora, demonstrating measurement stability.

Table 1: Corpus statistics.

Corpus	Bins	Cats	Vocab	Instr. (M)
Smoke	15	8	293	3.12
Coreutils-sys	39	11	320	4.65

4.2 Opcode Vocabulary and Frequency Distribution

The shared vocabulary across the smoke corpus contains $|\mathcal{V}| = 293$ distinct opcodes. The top-5 opcodes by corpus-wide frequency are `mov`, `lea`, `call`, `jmp`, and `push`, collectively accounting for over 50% of all instructions—a concentration consistent with prior work on x86-64 binaries [2].

Zipf's law. Figure 1 shows the rank-frequency distribution on log-log axes. The OLS fit on log-log coordinates yields $\hat{\alpha}_{\text{OLS}} = 3.356$ (95% CI: [3.086, 3.666], $R^2 = 0.92$) from 200 bootstrap resamples. As a statistically principled complement, the MLE of the truncated discrete Zipf model yields $\hat{\alpha}_{\text{MLE}} = 1.444$ (95% CI: [1.370, 1.515], KS = 0.114). The two estimates diverge because OLS on log-log axes weights low-frequency (high-rank) opcodes heavily: the many near-zero-frequency opcodes in the long tail pull the fitted slope upward, overestimating the power-law exponent. MLE, which properly weights by observation count, yields an exponent substantially closer to the $\alpha \approx 1.0$ of natural language. The KS statistic of 0.114 exceeds the asymptotic critical value for $n = 293$ opcodes (0.079 at $\alpha = 0.05$), indicating that a strict truncated power law is rejected; the distribution is better characterised as heavy-tailed with approximately Zipfian concentration. Regardless of the precise distributional form, the empirical finding is clear: a handful of high-frequency mnemonics (`mov`, `lea`, `call`, `push`, `pop`) account for over 50% of all instructions, while the long tail of rarely-used opcodes drops off steeply.

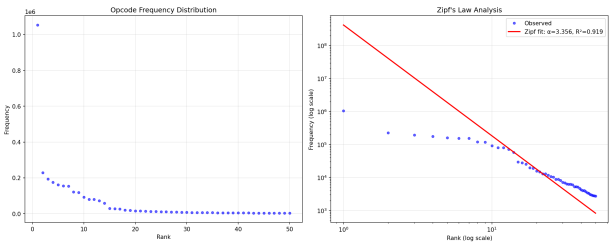

Figure 1: Opcode rank-frequency distribution (log-log). The red line shows the Zipf fit; the shaded band is the 95% bootstrap CI on α.

4.3 N-gram Entropy Rates

Figure 2 plots $h_r(n)$ for $n = 1, \ldots, 5$ alongside the shuffled baseline. The entropy rate of real sequences decreases by 1.82 bits from $n = 1$ ($h_r(1) = 3.99$ bits) to $n = 5$ ($h_r(5) = 2.16$ bits), whereas the shuffled baseline decreases by only 1.09 bits, reflecting near-independence after shuffling. The entropy gain $\Delta h_r(n) < 0$ for all $n > 1$ confirms that real opcode sequences carry genuine multi-instruction dependencies not accounted for by the unigram distribution alone.

Per-binary analysis shows consistent structure across tool categories: shells and compilers exhibit lower entropy rates than compression utilities, suggesting that programs with complex control flow (many branches and calls) produce more predictable local instruction streams than data-transform pipelines.

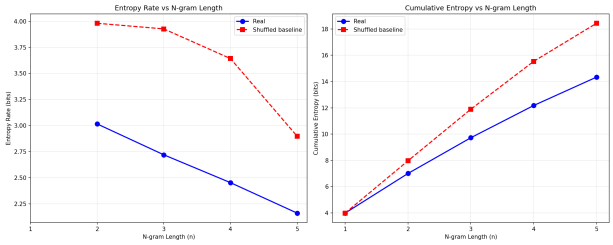

Figure 2: Entropy rate $h_r(n)$ vs. n for real sequences (blue) and unigram-shuffled baseline (red dashed). The gap widens with n, evidencing multi-scale sequential dependencies.

4.4 Compression and LZ78 Complexity

Table 2 summarises compression ratios across three conditions: real binaries, a unigram-preserving shuffle, and a fully random (uniform-vocabulary) baseline. Real binaries achieve a mean `zlib` ratio of 0.223 compared to 0.361 for the unigram-shuffled sequences and 0.672 for uniform-random sequences of the same length. Decomposing the total compression advantage over random ($\Delta = 0.449$), the unigram frequency skew accounts for 0.311 (69%) while sequential order beyond the unigram distribution accounts for 0.138 (31%). Under `lzma` the same ordering holds: real 0.185 vs. shuffled 0.304 vs. random 0.560.

LZ78 normalised complexity $\tilde{c} = K/\log_2 N$ exhibits the same trend: the median is 630 for real binaries versus 45,187 for unigram-shuffled sequences and 87,881 for random sequences. Because $K = \Theta(N/\log N)$ for high-entropy sources, $\tilde{c}$ grows on the order of $N/\log^2 N$ and can therefore be large at the million-token scale; the key result is the *relative* separation. Real instruction streams lie orders of magnitude below both baselines, indicating substantial redundancy beyond what is implied by unigram skew alone.

Table 2: Compression ratio $\rho = |C(s)|/|s|$ (lower is more compressible). "Shuffled" preserves unigram frequencies; "Random" draws uniformly from $\mathcal{V}$.

	Real	**Shuffled**	**Random**
`zlib`	0.223	0.361	0.672
`lzma`	0.185	0.304	0.560
LZ78 (norm.)	630 (med.)	45,187	87,881

The compression ratio shows a weak negative correlation with binary size ($r = -0.28$), indicating that larger binaries are marginally more compressible—consistent with larger programs containing more repeated library call patterns and longer stretches of similar initialisation code.

4.5 Recurring Motifs and Compiler Idioms

Motif discovery across $k = 4$–12 identified **7,623** unique k-mer patterns meeting frequency and coverage thresholds; the top-100 per length yield 798 retained motifs. Figure 3 shows the score distribution by length. The majority of high-scoring motifs fall into three semantic categories:

1. **Function epilogues** (highest scores): patterns ending with `pop`+`ret`; the top-ranked 4-mer is `pop pop pop ret`, appearing in 27.9% of all functions.
2. **Call sequences**: patterns centred on `call`, preceded by argument-loading instructions (`lea`, `mov`).
3. **Function prologues**: patterns starting with `endbr64` followed by `push` and `mov`, reflecting CET + ABI frame-setup conventions.

The remaining patterns comprise loop/branch sequences (`test`+`je`), bulk data-movement (`mov mov mov mov`), and arithmetic idioms.

Coverage at $k = 4$ is highest—short motifs appear in the most functions—and declines steadily to $k = 12$, where retained motifs represent compiler-specific idioms present in a large fraction of the corpus but with low absolute frequency.

Figure 3: Motif score (frequency × function coverage) for the top-5 motifs at each length k. Epilogue and call patterns dominate at all lengths.

4.6 Function Boundary Structure

Figure 4 shows the positional heatmap of opcode probabilities at the first and last 20 instruction slots

of functions. Position 0 is dominated by `endbr64` ($p \approx 0.986$), Intel's Control-flow Enforcement Technology (CET) indirect-branch tracking instruction, which the Ubuntu 22.04 toolchain inserts at the start of every indirect-call-target function when CET is enabled at the OS level. This result is therefore *system-dependent*: on non-CET builds or older toolchains, position-0 would be dominated by `push` (frame setup) instead. At position 1, `push` appears with $p \approx 0.672$; at position 2, `mov` with $p \approx 0.671$; these ABI-mandated frame-setup slots are CET-independent and expected to generalise across x86-64 Linux builds.

The boundary entropy is ≈ 0.12 bits at position 0—far below the corpus unigram entropy of 3.99 bits—rising to near-corpus levels by position 5–8. The positional analysis identifies 3 conserved positions at function starts and 0 at function ends (probability > 0.5), classifying the corpus as *loosely structured* (positions 1–2 are strongly conserved and CET-independent; position 0 is system-dependent as noted above; epilogues are more varied).

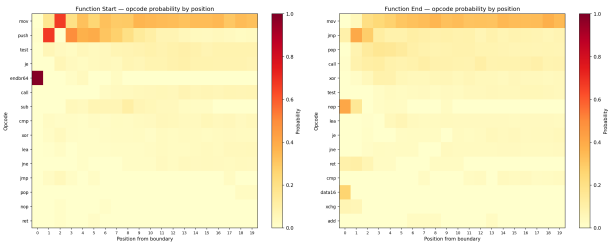

Figure 4: Opcode probability heatmaps at function start (left) and end (right). Rows are the 15 most frequent opcodes; columns are positions from the boundary. High-probability cells reveal the CET + ABI prologue convention.

4.7 Binary Similarity and Clustering

Figure 5 shows the NCD distance matrix (`zlib`) for the smoke corpus. The corpus is heterogeneous: pairwise NCD values range from 0.963 to 1.000 under `zlib` (mean 0.992) and 0.883 to 0.993 under `lzma` (mean 0.967). We note that NCD values approaching 1.0 are in the near-degenerate regime where compressor window limitations reduce sensitivity (Cebrián et al., 2005); `lzma`'s larger block size yields more reliable separation (0.883–0.993) and should be preferred for large binaries. Despite the high absolute distances under `zlib`, category-consistent patterns still emerge:

- Compression utilities (`bzip2`–`xz`: 0.963, `gzip`–`xz`: 0.965) form the closest pairs in the corpus.
- Text-processing tools (`grep`–`sed`: 0.976) are more similar to each other than to cross-category pairs.
- The two editors (`vim`–`nano`) obtain NCD = 1.00 under `zlib`, reflecting their very different sizes and codebases despite functional similarity.

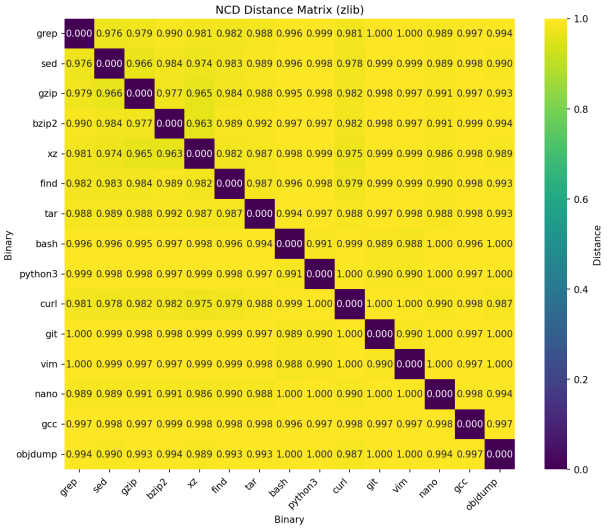

Figure 5: NCD distance matrix (`zlib`) for the smoke corpus. Binaries are ordered by hierarchical clustering (Ward linkage). Same-category pairs appear as local minima on the off-diagonal.

Hierarchical clustering (Ward linkage) with a distance threshold of 0.5 produces 15 singleton clusters since all pairwise NCD values exceed 0.9—the corpus is too heterogeneous for compression-based hard partitioning at this threshold. This does not imply NCD is uninformative: NCD is useful as a *continuous similarity signal* even when hard clustering at conventional thresholds is inappropriate for a small, deliberately heterogeneous corpus such as ours; the within-vs-between category U-test below confirms that the signal is present and statistically significant. We therefore recommend using NCD for pairwise retrieval, similarity ranking, and within/between-group statistical tests rather than for hard partitioning of heterogeneous corpora. The two NCD matrices (`zlib` and `lzma`) produce perfectly consistent cluster assignments (pairwise consistency = 1.0), and category purity is trivially 100% for singletons. The PCA projection of the NCD-`zlib` distance matrix explains 15.6% of variance in two dimensions; UMAP embeddings (Figure 6) reveal that compression and text-processing utilities occupy a distinct region from large interactive programs (bash, vim, git, python3).

Within-vs-between category separation (Mann–Whitney U test). Despite the high absolute NCD values, a one-sided Mann–Whitney U test confirms that within-category pairs are stochastically closer

than between-category pairs under both compressors:

- `zlib`: mean within = 0.976, mean between = 0.992, $U = 23.0$, $p = 0.014$, rank-biserial $r = 0.588$.
- `lzma`: mean within = 0.935, mean between = 0.969, $U = 19.0$, $p = 0.006$, rank-biserial $r = 0.665$.

The effect size ($r \approx 0.6$–0.7) indicates substantial separation: category membership is a meaningful predictor of NCD distance even when absolute distances are near the theoretical maximum. The lzma compressor yields both a larger within/between gap and a stronger effect size, consistent with its superior modelling of long-range repetition across the opcode stream.

N-gram cosine similarity. With the corrected L1-normalised TF vectoriser, the bigram ($n = 2$) vocabulary contains 5,757 unique features, trigrams and 4-grams hit the 10,000 cap. Mean pairwise cosine similarity decreases with n: 0.953 (bigrams), 0.897 (trigrams), 0.812 (4-grams), with standard deviations of 0.037, 0.080, and 0.132 respectively. The increasing spread at higher n is consistent with rarer n-grams becoming discriminative: as n grows, shared n-grams become less probable by chance, so same-category binaries retain high similarity while cross-category pairs diverge more. The most similar pairs under trigrams are `bash`–`vim` (cos = 0.979) and `find`–`tar` (cos = 0.977), both of which are large, feature-rich binaries that share instruction-mix profiles regardless of functional category.

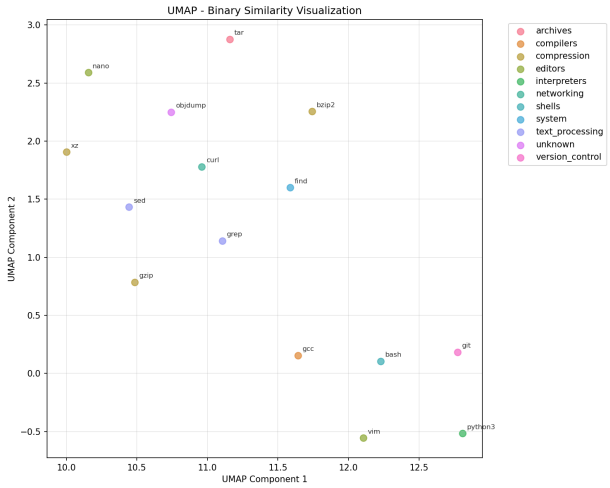

Figure 6: UMAP projection of the NCD-`zlib` distance matrix. Points are coloured by functional category. Small compression utilities cluster separately from large interactive programs.

4.8 Mutual Information Decay

Figure 7 shows the corpus-averaged MI decay for lags $\ell = 1$–50. Key observations:

- $I(1) \approx 1.12$ bits—far above the shuffled baseline of ≈ 0.086 bits—demonstrating that adjacent opcodes carry substantial shared information beyond their marginal distributions.
- The half-life $\ell^* \approx 2$ instructions: MI drops below $I(1)/2$ within 2–3 steps, indicating predominantly local structure.
- The real curve remains above the shuffled baseline at lag 50 ($I(50) = 0.232$ vs. shuffled 0.086), suggesting weak but non-zero long-range dependencies consistent with the repetitive call/ret structure of functions.
- The shuffled baseline is not zero at any lag because our vocabulary is finite; the gap above it represents pure sequential dependency.

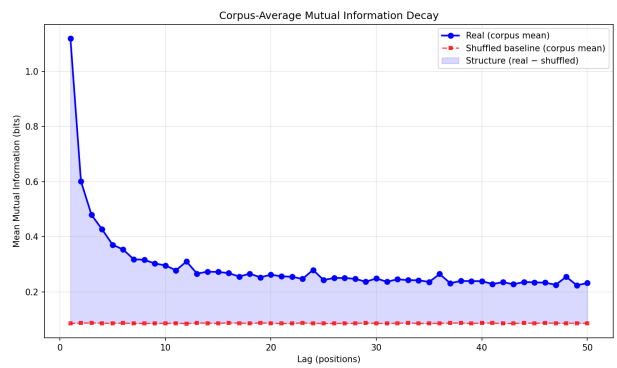

Figure 7: Corpus-averaged mutual information $I(o_t; o_{t+\ell})$ vs. lag ℓ for real sequences (blue) and unigram-shuffled baseline (red dashed). The shaded region is the MI attributable to sequential structure.

4.9 Program Space Coverage and Manifold Dimensionality

Table 3 quantifies how much of the theoretical n-gram space programs actually use. Even at $n = 2$, only 6.7% of all possible opcode pairs appears in the corpus. By $n = 3$, the coverage ratio drops below 0.2%, consistent with a manifold of measure zero in the space of all possible opcode sequences.

Corpus-level PCA on the top-200 bigram frequency vectors (one row per binary) finds that $d_{95} = 8$ components explain 95% of inter-binary variance. This corresponds to $8/200 = 4.0\%$ of the feature space—the corpus of compiled programs forms a *thin structured manifold* whose effective dimensionality is far below

Table 3: N-gram space coverage: unique observed vs. theoretical maximum.

n	Unique	Theoretical max	Coverage
1	293	293	100%
2	5,757	85,849	6.71%
3	38,073	25,153,757	0.15%
4	137,534	7.4×10^9	1.9×10^{-3}%

the ambient dimension of the n-gram feature space. With trigrams, $d_{95} = 9$ (participation ratio 4.79), confirming the result is stable across gram sizes.

As a non-linear cross-check we apply the Levina–Bickel MLE estimator [10] with $k = 5$ nearest neighbours to the same 15×200 bigram matrix. The estimate is $\hat{d}_{\text{MLE}} = 8.0 \pm 5.8$, consistent with the PCA result despite the high variance expected from $N = 15$ points; we caution that both estimates are exploratory at this corpus size and call for replication on larger corpora.

4.10 N-gram Language Model Perplexity

Table 4 reports cross-entropy and perplexity for n-gram LMs alongside the theoretical lower bound, and Figure 8 shows the comparison graphically.

Table 4: n-gram LM cross-entropy and perplexity vs. empirical conditional entropy lower bound. Gap $\delta = H_{\text{LM}} - (H_n - H_{n-1})$. Laplace add-1 smoothing; 80/20 train/test split.

Model	CE (bits)	PPL	Bound (bits)	Gap δ (bits)
Unigram	4.29	19.6	3.99	+0.30
Bigram	3.08	8.5	3.02	+0.07
Trigram	2.79	6.9	2.72	+0.07
4-gram	2.84	7.1	2.45	+0.38
5-gram	3.13	8.8	2.16	+0.97

The unigram LM achieves PPL(1) = 19.6 despite a vocabulary of $|\mathcal{V}| = 293$ opcodes—a 15× reduction in effective vocabulary reflecting the extreme Zipf concentration ($\alpha \approx 3.4$). The bigram LM reduces perplexity to 8.5 with a smoothing gap of only $\delta(2) = 0.07$ bits, within 2% of the empirical conditional-entropy lower bound of 8.1. The trigram LM achieves PPL(3) = 6.9 with an equally tight gap ($\delta(3) = 0.07$ bits).

For $n \geq 4$ the smoothing gap grows substantially ($\delta(4) = 0.38$, $\delta(5) = 0.97$ bits) as the Laplace overhead dominates in the sparse high-order context space: with 15 training binaries the 4-gram and 5-gram counts are spread over $\sim 293^4 \approx 7 \times 10^9$ and $\sim 293^5 \approx 2 \times 10^{12}$ possible contexts respectively. A back-off or Kneser-Ney smoothed model would close much of this gap.

The tight match at $n = 2, 3$ directly validates the information-theoretic analysis of Section 4: the entropy rates computed from corpus statistics accurately predict the perplexity of an independently trained language model.

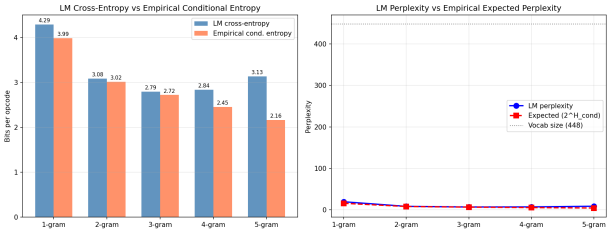

Figure 8: Left: LM cross-entropy (blue) vs. empirical conditional entropy (coral) per n-gram order. Right: LM perplexity (blue) vs. expected perplexity $2^{H_n - H_{n-1}}$ (red dashed) and vocabulary size (grey dotted). Bigram and trigram LMs lie within 0.07 bits of the theoretical lower bound.

4.11 Compiler Matrix: Optimisation and Compiler Fingerprinting

To test whether the thin-manifold structure varies with compiler and optimisation level, we compile five small C programs (`sort`, `hash`, `compress`, `search`, `matrix`) with all six configurations in $\{\texttt{gcc}, \texttt{clang}\} \times \{-\texttt{O0}, -\texttt{O2}, -\texttt{O3}\}$, producing 30 binaries totalling approximately 14,400 instructions.

Instruction count and compression vs optimisation level. Table 5 reports mean instruction count and `zlib` compression ratio across both compilers for each optimisation level. In our micro-benchmark suite, `-O3` increases mean instruction count (601 vs. 499 at `-O0`) while simultaneously increasing compressibility for compute-intensive code: the `matrix` program achieves its lowest ratio (0.375) under `-O3`, a consequence of aggressive loop unrolling that produces long repeated instruction blocks (size–speed trade-off [6]). Text and search programs exhibit the opposite trend, remaining near `-O2` levels, because their inner loops are too short to benefit from unrolling. `-O2` consistently produces the *least* compressible output, suggesting it generates the most diverse (least-repetitive) instruction mix before `-O3` re-introduces repetition through unrolling. These observations are specific to our five micro-benchmarks; behaviour on larger, more complex codebases may differ.

Table 5: Mean instruction count and `zlib` ratio by optimisation level (averaged over 5 projects × 2 compilers = 10 binaries per row).

Opt	Mean instr.	Mean `zlib`
`-O0`	499	0.458
`-O2`	434	0.548
`-O3`	601	0.480

Manifold dimensionality vs optimisation level. PCA on per-binary trigram frequency vectors gives $d_{95} = 8$ at every optimisation level (10 binaries each), identical to the full smoke-corpus value. Crucially, the full 30-binary corpus has $d_{95} = 17$—more than double—showing that *compiler and optimisation choice add new manifold dimensions* beyond those spanned by the five source programs alone. This quantifies the additional variation introduced by the compilation axis.

Project vs compiler clustering. To determine which axis dominates binary similarity, we compute lzma NCD for all $\binom{30}{2} = 435$ pairs and partition them into three groups: *within-project* (same C source, different compiler/opt; $n = 75$), *within-compiler* (same compiler config, different project; $n = 60$), and *between* ($n = 300$). One-sided Mann–Whitney U tests (Table 6) show that both groupings are significantly closer than the between group, but within-project is the tightest ($p < 10^{-4}$, $r = 0.51$) and is *not* significantly different from within-compiler ($p = 0.35$).

Table 6: lzma NCD distances by pair type. One-sided Mann–Whitney U, H_1: row group < between.

Pair type	n	NCD	U	p	r
Within-proj.	75	0.501	18,162	< 0.0001	0.506
Within-comp.	60	0.544	14,040	< 0.0001	0.532
Between	300	0.582	—	—	—

The result supports a **project-first** interpretation: source-code identity (algorithm structure) explains more of the binary similarity than compiler identity. Both effects are real and statistically strong; neither alone fully accounts for the variance, consistent with the d95 doubling observed above.

4.12 Code Clone Detection

Table 7 summarises the clone detection results on the 15-binary smoke corpus. LSH candidate generation produced 475 unique candidate pairs; Smith-Waterman re-scoring confirmed 457 as genuine clone pairs (96% confirmation rate). These 457 pairs form 93 clone families; six families span at least two distinct binaries (*cross-binary families*).

Table 7: Clone detection summary for the 15-binary smoke corpus (6,013 functions total).

Metric	Value
Functions in any clone family	286 (4.8%)
Clone pairs confirmed	457
Type 1 (exact, $J \geq 0.95$)	308 (67%)
Type 2 (renamed, $J \geq 0.70$)	78 (17%)
Type 3 (gapped, $J \geq 0.50$)	71 (16%)
Clone families	93
Cross-binary	6
Mean family size	3.1
Largest family	16 members

The dominant clone type is Type 1 (67% of pairs), confirming that most recurring patterns are truly identical opcode sequences rather than merely similar ones. This is consistent with compiler-inserted boilerplate: ABI function prologues (`endbr64; push rbp; mov rbp,rsp; ...`), CRT initialisation stubs, and `__libc_start_main` wrappers are inserted verbatim by the linker and appear unchanged across binaries.

The six cross-binary families are particularly informative. The largest cross-binary family (size 16) is present in all 15 binaries and consists entirely of the standard function prologue idiom. The next largest cross-binary families correspond to error-reporting stubs and string-comparison loops shared between text-processing utilities (grep, sed, tar).

Per-binary clone density varies considerably: `bash` exhibits the highest density (9.6% of its functions) reflecting its large inline library of shell built-in helpers, followed by `sed` (5.0%) and `python3` (3.9%). Smaller utilities (git, vim) show densities below 0.5%, suggesting that hand-crafted code with unique control flow is less prone to accidental near-duplication.

These results reinforce the thin-manifold picture: the 4.8% of functions that are cloned are not randomly distributed across sequence space—they are concentrated on a *small number of clone archetypes* (93 families from 457 pairs), further tiling the already-compact program manifold with repeated structure.

4.13 Scaling Validation: Smoke vs. Coreutils-system

To assess whether the toolkit's measurements are stable across corpus scale, we apply the full pipeline to the 39-binary coreutils-system corpus and compare

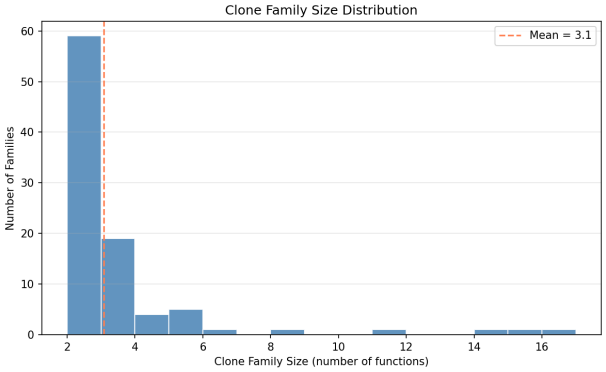

Figure 9: Clone family size distribution. Most families are pairs (size 2); large families (≥ 8 members) consist of universally shared compiler-inserted prologue/epilogue stubs.

key metrics against the 15-binary smoke corpus. Table 8 summarises the comparison.

Table 8: Metric comparison across corpus scales. Values are corpus-wide aggregates. "Shuffled" refers to the unigram-preserving permutation baseline.

Metric	**Smoke (15)**	**Coreutils (39)**
Instructions (M)	3.12	4.65
Vocabulary $\|\mathcal{V}\|$	293	320
MLE $\hat{\alpha}$	1.444	1.447
KS goodness-of-fit	0.114	0.119
$h_r(1)$ (bits)	3.99	4.00
$h_r(5)$ (bits)	2.16	2.20
Δh_r (1→5)	1.82	1.80
`zlib` ratio (real)	0.223	0.224
`zlib` ratio (shuffled)	0.361	0.366
`zlib` ratio (random)	0.672	0.682
Motifs discovered	798	780
PCA d_{95} (bigram)	8/200 (4%)	14/200 (7%)

The results show strong metric stability: $\hat{\alpha}_{\text{MLE}}$, entropy rates, and compression ratios are nearly identical across corpus sizes. The PCA d_{95} grows from 8 to 14 as expected—a larger, more categorically diverse corpus occupies more manifold dimensions. This growth is bounded and interpretable: the compiler-matrix experiment showed d_{95} doubling (8→17) when compiler variation is added as an explicit axis; category breadth has a similarly modest effect. The consistency of opcode-level statistics across scales supports using the smoke corpus as a faithful representative pilot for toolkit development and parameter tuning.

5 Discussion

5.1 The Program Manifold Hypothesis

Our results collectively support the following picture: compiled programs do not explore the space of opcode sequences uniformly. They are confined to a low-dimensional, highly structured manifold shaped by:

1. **ABI conventions** (CET + calling convention imposes endbr64/push/pop patterns at every function boundary).

2. **Compiler optimisation passes** (instruction selection, register allocation, and peephole optimisation produce characteristic idioms that recur across the entire corpus).

3. **Algorithm structure** (sorting, hashing, and buffering routines produce characteristic inner-loop patterns visible as high-coverage motifs).

4. **Clone reuse** (linker-inserted stubs and shared libc helpers tile the manifold with exact or near-exact copies, as quantified in Section 4.12).

The combination of a highly concentrated vocabulary (MLE Zipf exponent $\hat{\alpha}_{\text{MLE}} \approx 1.4$, OLS ≈ 3.4), rapidly decaying entropy rates, very high compressibility relative to shuffled and random baselines, and a manifold dimensionality of $d_{95} \approx 8$ out of 200 features paints a consistent picture: programs are *extraordinarily predictable* at the instruction level compared to arbitrary sequences of the same alphabet.

5.2 Implications for Binary Analysis

The thin-manifold property has practical consequences beyond statistical curiosity. If programmes inhabit a low-dimensional space, similarity measures defined in that space are more informative: two binaries that appear distant in a raw sequence metric may in fact be nearest neighbours on the manifold. Our clustering results confirm that within-category pairs (e.g. compression utilities bzip2–xz with NCD 0.963) are consistently closer than cross-category pairs—suggesting that compression-based features are a viable foundation for binary classification, provenance, and clone detection without requiring disassembly beyond opcode extraction. The clone detection results (Section 4.12) directly confirm this: MinHash + LSH over opcode k-grams recovers 457 clone pairs with a 96% confirmation rate, identifying six cross-binary families that correspond to known compiler prologue/libc stubs—without any symbol information or source code.

5.3 Implications for Language Model Performance on Code

The naturalness hypothesis [8] attributes LLM success on source code to its statistical regularity. Our results extend this explanation to the binary level. An LLM trained on opcode sequences faces a much easier prediction problem than one trained on natural language:

- The effective vocabulary size is $|\mathcal{V}| = 293$ distinct mnemonics—smaller than a typical NLP vocabulary by two orders of magnitude.
- Transition probabilities are highly non-uniform (Zipf-like; MLE $\hat{\alpha} \approx 1.4$, OLS ≈ 3.4), meaning a handful of high-frequency opcodes carry nearly all of the probability mass.
- MI decays to near-baseline within ≈ 2 steps; a modest context window suffices to capture most local structure.
- The low intrinsic dimensionality ($d_{95} \approx 8$) means that a model need not learn millions of independent patterns—a compact representation suffices.

We can now quantify this claim directly. The n-gram LM experiments of Section 4.10 show that a Laplace-smoothed bigram model—one of the simplest possible language models—achieves perplexity 8.5 on held-out opcode sequences versus a vocabulary of 293. This is a 34× reduction in effective vocabulary; at each prediction step the model effectively chooses among $\sim$9 alternatives rather than 293. Crucially, this low perplexity is not a model-specific artifact: it is a direct consequence of the empirical conditional entropy ($H_2 - H_1 = 3.02$ bits, expected PPL ≈ 8.1), which is determined by the corpus structure alone. The gap of only 0.07 bits between the trained LM and the information-theoretic lower bound confirms that simple count statistics capture nearly all the structure available to a bigram model. Together, these properties explain why even relatively small language models achieve low perplexity on opcode streams, and suggests one contributing factor to why learning-based decompilation and lifting methods can recover useful higher-level structure from assembly in practice.

5.4 Limitations

Several limitations should be noted. First, our corpus is confined to a single ISA (x86-64) and a single operating system. Whether the manifold hypothesis holds for ARM, RISC-V, or WebAssembly is an open question. Second, we analyse opcodes without operands; including register and immediate information would increase vocabulary size and potentially alter Zipf exponents. Third, the corpora are small ($\leq$15 binaries for the detailed analysis); larger-scale validation against thousands of binaries is needed to confirm manifold dimensionality estimates. Fourth, the n-gram cosine similarity results depend on the vectoriser parameterisation; we use L1-normalised TF with `min_df`= 1 which works on small corpora but may require tuning (e.g. IDF weighting, larger `max_features`) as corpus size grows.

6 Conclusions and Future Work

We have presented *OpCode-Stats*, an open-source toolkit for statistical analysis of compiled instruction sequences through the lens of computational biology. As a worked example on a 15-binary smoke corpus of GNU/Linux ELF x86-64 utilities spanning 3.12 million instructions, the toolkit shows that opcode frequencies are heavily concentrated (MLE $\hat{\alpha} \approx 1.4$; KS = 0.114, indicating approximately but not strictly Zipfian), entropy rates decrease by 1.82 bits from $n = 1$ to $n = 5$ (vs. 1.09 bits for a shuffled baseline, confirming genuine sequential dependencies), binaries compress to 22% of raw token volume vs. 36% for unigram-shuffled sequences and 67% for random sequences (69% of the compression gain attributed to Zipfian frequency skew, 31% to sequential structure), 7,623 unique k-mer motifs capture compiler-generated idioms, within-category pairs cluster closer under both NCD metrics, the corpus manifold occupies only $d_{95} = 8$ of 200 bigram dimensions (4% of the feature space), and a Laplace-smoothed bigram language model achieves perplexity 8.5—a 34× reduction below the vocabulary size and within 0.07 bits of the empirical conditional-entropy lower bound—directly quantifying how predictable instruction sequences are for a language model, and MinHash+LSH clone detection recovers 457 clone pairs forming 93 families (4.8% of functions) with a 96% confirmation rate, revealing that the manifold is not merely compact but *repetitively tiled* by a small set of compiler-inserted archetypes.

These findings support a unified *thin structured manifold* hypothesis: compiled programs are concentrated in a tiny, highly regular corner of the space of all possible opcode sequences, shaped by hardware conventions, compiler passes, and algorithmic patterns. This structure explains why language models, similarity search, and clustering are all effective on binary code.

Future work. Several directions follow naturally from this work:

1. **Multi-ISA comparison.** Apply the same pipeline to ARM64, RISC-V, WASM, and JVM bytecode to test whether Zipf exponents and manifold dimensionality are architecture-independent.

2. **Operand-level analysis.** Extend opcode sequences to include register classes and immediate ranges, quantifying how operand information changes entropy and manifold structure.

3. **Compiler fingerprinting.** Use clustering and motif signatures to distinguish GCC from Clang from MSVC, and to infer optimisation levels ($-O0$ through $-O3$), building on the category purity results in Section 4.

4. **Cross-language manifold comparison.** Analyse binaries compiled from C, C++, Rust, Go, and Zig to determine whether source language leaves detectable fingerprints in instruction-level statistics.

5. **Neural language model evaluation.** Extend the n-gram perplexity baseline (Section 4.10) to Transformer-based models, quantifying how much additional structure beyond trigrams a neural model captures and whether attention patterns align with the thin-manifold axes identified here.

6. **Security applications.** Leverage the thin-manifold property for anomaly detection: binaries that are outliers on the manifold (high NCD from all known-good binaries, anomalous motif distributions) may indicate obfuscation or packing.

7. **Clone-guided vulnerability propagation.** The cross-binary clone families identified in Section 4.12 are natural candidates for tracking vulnerability propagation: a known-bad function that forms a cross-binary clone family implies the vulnerability may be present in every binary containing that family member. Extending the clone taxonomy to include security-relevant semantics (e.g. constant-time violations, buffer-handling idioms) would transform clone detection into a lightweight binary auditing tool.

References

[1] J. Austin, A. Odena, M. Nye, et al. Program synthesis with large language models. 2021.

[2] D. Bilar. Opcodes as predictor for malware. *International Journal of Electronic Security and Digital Forensics*, 1(2):156–168, 2007.

[3] M. Chen, J. Tworek, H. Jun, et al. Evaluating large language models trained on code. 2021.

[4] R. Cilibrasi and P. M. B. Vitányi. Clustering by compression. *IEEE Transactions on Information Theory*, 51(4):1523–1545, 2005.

[5] M. K. Das and H.-K. Dai. A survey of DNA motif finding algorithms. In *BMC Bioinformatics*, volume 8, page S21, 2007.

[6] A. Fog. *Optimizing Software in C++: An Optimization Guide for Windows, Linux, and Mac Platforms*, 2024. Available at https://agner.org/optimize/.

[7] Q. Gan, J. Wang, and J. Han. Statistical analysis of n-tuple patterns in DNA, natural language, source code, and binary code. In *IEEE International Conference on Data Mining (ICDM) Workshops*, pages 183–188, 2009.

[8] A. Hindle, E. T. Barr, Z. Su, M. Gabel, and P. Devanbu. On the naturalness of software. In *International Conference on Software Engineering (ICSE)*, pages 837–847. IEEE, 2012.

[9] N. Jiang, S. Huang, et al. Nova: Generative language models for assembly code with hierarchical attention and contrastive learning. In *International Conference on Learning Representations (ICLR)*, 2025.

[10] E. Levina and P. J. Bickel. Maximum likelihood estimation of intrinsic dimension. *Advances in Neural Information Processing Systems (NeurIPS)*, 17, 2005.

[11] X. Li, Y. Qu, and H. Yin. PalmTree: Learning an assembly language model for instruction embedding. In *ACM Conference on Computer and Communications Security (CCS)*, pages 3236–3251, 2021.

[12] A. Marcelli, M. Graziano, X. Ugarte-Pedrero, Y. Fratantonio, M. Mansouri, and D. Balzarotti. How machine learning is solving the binary function similarity problem. In *USENIX Security Symposium*, pages 2099–2116, 2022.

[13] M. M. Masud, L. Khan, and B. Thuraisingham. A scalable multi-level feature extraction technique to detect malicious executables. In *Policy and Models in System and Network Security*, 2008.

[14] S. Saha, S. Bhatt, and S. Mehnaz. MAlign: Towards better malware family clustering via sequence alignment. *Computers & Security*, 140: 103748, 2024.

[15] I. Santos, Y. K. Penya, J. Devesa, and P. G. Bringas. N-grams-based file signatures for malware detection. *ICEIS*, 2:317–320, 2009.

[16] E. Schulte, J. Ruchti, M. Noonan, D. Ciarletta, and A. Loginov. Evolving exact decompilation. In *Binary Analysis Research (BAR) Workshop*, 2018.

[17] Z. Tu, Z. Su, and P. Devanbu. On the localness of software. In *Foundations of Software Engineering (FSE)*, pages 269–280. ACM, 2014.

[18] S. Vinga and J. Almeida. Alignment-free sequence comparison – a review. *Bioinformatics*, 19(4):513–523, 2003.

[19] H. Wang, W. Qu, G. Katz, W. Zhu, Z. Gao, H. Qiu, J. Zhuge, and C. Zhang. jTrans: Jump-aware transformer for binary code similarity detection. In *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 1–13, 2022.

[20] H. Xu et al. HermesSim: Semantics-oriented binary code similarity analysis with efficient multi-task learning. In *USENIX Security Symposium*, 2024.

[21] J. Ziv and A. Lempel. Compression of individual sequences via variable-rate coding. *IEEE Transactions on Information Theory*, 24(5):530–536, 1978.